Context Engineering with Memory Bank

Module 3

February 2026

Context Engineering with Memory Banks

- **Module Focus:** Teaching AI assistants about YOUR project
- **Key Question:** How do we give AI persistent, project-specific knowledge?
- **Practical Outcome:** Configure memory banks that improve AI accuracy by 40-60%

Journey So Far: Precision → Structure → Context

- **Module 01:** Precision Principle - Vibe coding fails, specs succeed
- **Module 02:** Specification Hierarchy - PRD → Epic → Story → Agents.MD
- **Module 03 (Current):** Context Engineering - Teaching AI about YOUR project

Running Example: Task Manager (From Module 02)

We'll use the **Team Task Manager** project from Module 02 as our running example:

- 45-person engineering team across 4 time zones
- GitHub and Teams integration
- Replacing heavyweight Jira with lightweight tool **Why This Example?**
- You already created PRD, Epics, Stories for this
- Now we'll configure memory banks to guide AI implementation

What We'll Cover in This Module

Part 1: The Context Problem (45 min)

- Why AI forgets between sessions
- Token limits and context windows
- Real-world context failures

Part 2: Memory Bank Fundamentals (50 min)

- What are memory banks?
- Types of memory (episodic, semantic, procedural, working)
- Memory bank architecture

Part 3: Configuring Memory Banks (30 min)

- Live demo: Setting up project memory
- Role-specific memory banks
- Integration with Agents.MD

Part 4: Hands-On Lab (95 min)

- Create memory banks for your project
- Test AI outputs with/without memory
- Peer review and refinement

Why AI "Forgets" Your Project

The Problem: AI coding assistants are **stateless by default**—each session starts fresh.

Example Conversation:

Session 1 (Monday 9am):

- You: "Generate API endpoint for task creation following our microservices pattern"
- AI: *Generates code using your architecture*

Session 2 (Tuesday 9am):

- You: "Generate API endpoint for task updates"
- AI: *Generates code… but uses DIFFERENT architecture!*

Why? AI doesn’t remember previous conversations.

The Technical Reality: Context is Expensive

Context Window Limits:

Model	Context Window	Cost Implication
GPT-4 Turbo	128K tokens	~$10 per 1M input tokens
Claude Sonnet 4.5	200K tokens	~$3 per 1M input tokens
Gemini Pro	1M tokens	~$1.25 per 1M input tokens

Practical Limit:

- Large codebase: 500K+ tokens
- Entire context won't fit in one conversation

The Challenge:

- What context do we include?
- What can we omit?
- How do we organize for efficient retrieval?

What Happens Without Context?

Scenario: Developer asks AI to generate authentication middleware

Without Memory Bank:

```
// AI generates generic Express middleware
export function authenticate(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'No token' });

  // Uses JWT (but team uses OAuth2!)
  jwt.verify(token, process.env.SECRET, (err, decoded) => {
    if (err) return res.status(403).json({ error: 'Invalid token' });
    req.user = decoded;
    next();
  });
}
```

Problems:

- ❌ Team uses **OAuth2**, not JWT
- ❌ Wrong error response format (team uses standardized error structure)
- ❌ Missing **logging** (team requirement)
- ❌ Not using team's **auth service client**

With Memory Bank:

```
// AI generates code aligned with project standards
import { OAuth2Client } from '@/lib/auth/oauth2-client';
import { logger } from '@/lib/logger';
import { ErrorResponse } from '@/types/api';

export async function authenticate(req: Request, res: Response, next: NextFunction) {
  try {
    const authHeader = req.headers.authorization;
    if (!authHeader?.startsWith('Bearer ')) {
      return res.status(401).json(
        ErrorResponse.unauthorized('Missing or invalid authorization header')
      );
    }

    const token = authHeader.substring(7);
    const user = await OAuth2Client.validateToken(token);

    req.user = user;
    logger.info('User authenticated', { userId: user.id, path: req.path });
    next();
  } catch (error) {
    logger.error('Authentication failed', { error });
    return res.status(401).json(ErrorResponse.unauthorized('Invalid token'));
  }
}
```

Why Better:

- ✅ Uses OAuth2Client (team standard)
- ✅ Standardized ErrorResponse format
- ✅ Includes logging (team requirement)
- ✅ TypeScript types (team convention)

Quantifying the Impact

Without Memory Banks:

- **Time lost per developer:** 15-30 min/day explaining context
- **Code rework:** 20-30% of AI-generated code needs correction
- **Inconsistency:** Each developer gets different AI interpretations
- **Cognitive load:** Constantly switching between coding and explaining

Example Calculation (10-developer team):

- 10 developers × 20 min/day × 20 working days = **67 hours/month**
- At $100/hour blended rate = **$6,700/month wasted**
- **$80,400/year** in context overhead

With Memory Banks:

- **Setup time:** 2-4 hours (one-time)
- **Maintenance:** 1-2 hours/month
- **ROI:** Positive within first week

What is Context Engineering?

Definition: Context engineering is the **systematic design and management of context** provided to AI systems to improve output quality, consistency, and relevance.

Three Pillars:

1. **Structure** - How context is organized
   - Hierarchical (general → specific)
   - Modular (separable concerns)
   - Versioned (track changes)
2. **Content** - What context includes
   - Architecture decisions
   - Coding conventions
   - Domain knowledge
   - Project history
3. **Delivery** - When/how context is provided
   - Persistent (memory banks)
   - On-demand (retrieval)
   - Progressive (relevant context for current task)

Group Discussion: When Has AI Failed You?

Think about a time when AI generated code that was "technically correct" but wrong for YOUR project.

Discussion Prompts:

1. What was the task?
2. What did AI generate?
3. What was wrong with it?
4. How much time did it take to correct?
5. What context was missing?

Format:

- 2 minutes: Think individually
- 5 minutes: Share in pairs or small groups
- 3 minutes: Volunteers share with full group

Goal: Identify common context failure patterns

Memory Banks: Persistent Context for AI

Definition: Memory banks are **structured knowledge repositories** that provide AI assistants with persistent, project-specific context across sessions.

Key Characteristics:

Feature	Description
Persistent	Context survives across sessions (unlike chat history)
Structured	Organized in modular, hierarchical format
Selective	Only relevant context loaded per task
Versioned	Track changes as project evolves
Role-aware	Different context for Dev vs QA vs PM

Think of memory banks as:

- **Project handbook** that AI reads before every conversation
- **Onboarding guide** for a new AI teammate
- **Institutional knowledge** encoded for machines

Four Types of Memory in AI Systems

1. Episodic Memory

- **What:** Specific past interactions and events
- **Example:** "Last time user asked about auth, we discussed OAuth2"
- **Use Case:** Referencing previous conversations

2. Semantic Memory

- **What:** General knowledge and facts about the project
- **Example:** "This project uses microservices with PostgreSQL"
- **Use Case:** Architecture, tech stack, domain concepts

3. Procedural Memory

- **What:** "How to" knowledge - processes and patterns
- **Example:** "When adding API endpoint: 1) Define schema, 2) Write controller, 3) Add tests"
- **Use Case:** Workflows, conventions, coding patterns

4. Working Memory

- **What:** Current task context (temporary)
- **Example:** "Currently working on Epic 2, Story 2.3"
- **Use Case:** Active task context, recent decisions

Our Focus: Semantic and Procedural memory (most impactful for teams)

Recommended Structure

Key Principles:

- **Modular:** Each file has single responsibility
- **Hierarchical:** General (architecture) → Specific (conventions)
- **Discoverable:** Clear navigation via README

```
memory-banks/
├── README.md                    # Index and navigation
├── architecture/
│   ├── overview.md              # System architecture
│   ├── decisions/               # ADRs (from Module 05)
│   └── diagrams/                # Architecture diagrams
├── conventions/
│   ├── coding-standards.md      # From Agents.MD (Module 02)
│   ├── testing-patterns.md
│   ├── naming-conventions.md
│   └── error-handling.md
├── domain/
│   ├── glossary.md              # Domain terms
│   ├── business-rules.md
│   └── user-personas.md         # From PRD (Module 02)
├── workflows/
│   ├── development-process.md
│   ├── code-review-checklist.md
│   └── deployment-process.md
├── [illegible]s/
│   developer.md                 # Dev-specific context
│   ├── qa.md                    # QA-specific context
```

Content Guidelines

DO Include: ✅ Architecture patterns and decisions ✅ Tech stack and library versions ✅ Coding conventions and style guides ✅ Testing strategies and coverage requirements ✅ Domain glossary and business rules ✅ Common workflows (dev, review, deploy) ✅ Known issues and workarounds ✅ Integration points and APIs

DON'T Include: ❌ Secrets, credentials, API keys (use environment variables) ❌ PII (Personally Identifiable Information) ❌ Entire codebase (too large, changes frequently) ❌ Generated documentation (use source of truth) ❌ Overly detailed implementation (AI can infer) ❌ Outdated information (maintain currency)

Goldilocks Principle:

- Too little context → Generic AI outputs
- Too much context → Cognitive overload, high costs
- **Just right context** → Accurate, efficient, cost-effective

When to Use Each?

Comparison:

Mechanism	Scope	Persistence	Best For	Example
Memory Banks	Project-wide	Permanent	Architecture, domain knowledge, workflows	"Our system uses microservices"
Agents.MD	Project/Repo-specific	Permanent	Coding standards, quality criteria, tech stack	"Use kebab-case for files"
Inline Prompts	Task-specific	Temporary	Specific instructions for current task	"Generate API endpoint for user login"

Relationship:

- **Memory Banks** provide broad context (the "world")
- **Agents.MD** provides coding rules (the "how")
- **Inline Prompts** provide task details (the "what")

Use Together: Memory Bank (architecture) + Agents.MD (standards) + Inline Prompt (task) = High-quality AI output

Available Tools (2025)

Memory Bank Implementations:

1. Cline + Memory Bank (VS Code Extension)

- **Setup:** Memory banks as markdown files in `.cline/memory/`
- **Auto-loading:** Loaded automatically when Cline starts
- **Best For:** VS Code users, individual developers

2. GitHub Copilot + Workspace Context

- **Setup:** Context files in `.github/copilot/`
- **Integration:** Workspace-wide context
- **Best For:** Teams using GitHub Copilot

3. Claude Code + Project Instructions

- **Setup:** `.claude/` directory with memory files
- **Approach:** Project-specific instructions and knowledge
- **Best For:** Teams using Claude Code

4. Custom RAG (Retrieval-Augmented Generation)

- **Setup:** Vector database (Pinecone, Weaviate) + embedding model
- **Complexity:** Higher, requires infrastructure
- **Best For:** Enterprise, large codebases

Note: Exact tool names and APIs evolve rapidly. Check current documentation.

What Are Skills?

Definition: Skills are **reusable prompt templates** stored as project files that encode repeatable workflows, best practices, and domain-specific instructions for AI assistants.

Key Characteristics:

Feature	Description
Reusable	Invoke the same workflow across tasks with a slash command
Parameterized	Accept arguments to customize behavior per invocation
Shareable	Committed to repo — entire team uses the same skills
Composable	Skills can reference memory banks and other context
Versioned	Evolve with git alongside your codebase

Think of skills as:

- **Macros** for your AI assistant — one command triggers a complex workflow
- **Runbooks** encoded as prompts — tribal knowledge that anyone can invoke
- **Recipes** — step-by-step instructions the AI follows precisely

How Skills Are Organized

Directory Structure (Claude Code Example):

```
.claude/
├── CLAUDE.md                    # Project instructions (Agents.MD)
├── commands/                    # ← Skills live here
│   ├── review-pr.md             # /review-pr — code review workflow
│   ├── generate-tests.md        # /generate-tests — test generation
│   ├── create-adr.md            # /create-adr — ADR from template
│   ├── decompose-story.md       # /decompose-story — epic → stories
│   └── debug-issue.md           # /debug-issue — structured debugging
└── settings.json                # Tool configuration
```

Anatomy of a Skill File:

```
# /review-pr — Code Review Skill

Review the specified pull request against our team standards.

## Steps
1. Read the diff and identify changed files
2. Check against coding conventions in memory bank
3. Verify test coverage for new code paths
4. Flag security concerns (OWASP Top 10)
5. Summarize findings with severity ratings

## Output Format
itical | 🟡 Warning | 🟢 Suggestion
```


Type `/review-pr 42` → AI executes the full workflow

Skill Categories for Your Team

1. Specification Skills

Skill	What It Does
`/create-prd`	Generate PRD from problem statement (Module 02 format)
`/decompose-story`	Break epic into testable stories with acceptance criteria
`/create-adr`	Write ADR from decision context using team template

2. Development Skills

Skill	What It Does
`/generate-tests`	Create tests from acceptance criteria (BDD-style)
`/implement-story`	Generate implementation from story + ADR context
`/refactor`	Refactor code while preserving behavior and tests

3. Quality & Review Skills

Skill	What It Does
`/review-pr`	Full code review against team standards
`/security-check`	OWASP Top 10 scan on changed files
`/check-coverage`	Verify test coverage meets team threshold

4. Knowledge Skills

	What It Does
`/update-memory`	Update memory bank after architectural change

Principles for Good Skills

✅ **Do:**

Principle	Example
Be specific about steps	"1. Read the story file 2. Extract acceptance criteria 3. Generate one test per criterion"
Define output format	"Return a markdown table with columns: Finding, Severity, Suggestion"
Reference context sources	"Load architecture from memory bank before generating"
Include guard rails	"If no acceptance criteria found, ask the user before proceeding"
Keep skills focused	One skill = one workflow (don't combine review + fix + deploy)

❌ **Don't:**

Anti-Pattern	Why It Fails
Vague instructions	"Review the code and give feedback" — no structure, inconsistent results
Mega-skills	500-line skill that does everything — AI loses focus, output quality drops
Hardcoded values	"Check file `src/auth/login.ts`" — breaks when code moves
No output format	AI returns different formats each time — can't automate downstream

The Goldilocks Rule: A good skill is **20-60 lines** — enough structure to be consistent, enough flexibility for AI to adapt to the specific situation.

The Context Engineering Stack

How everything fits together:

Layer	What It Provides	Persistence	Example
Memory Banks	Project knowledge (the "world")	Permanent	"We use microservices with PostgreSQL"
Agents.MD / CLAUDE.md	Coding rules (the "how")	Permanent	"Use TypeScript strict mode, 80% coverage"
Skills	Reusable workflows (the "actions")	Permanent	`/review-pr` — structured code review
Inline Prompts	Task-specific details (the "what")	Temporary	"Implement login endpoint for Story 2.3"

The Flow:

1. **Memory Banks** tell AI about the project architecture and domain
2. **Agents.MD** tells AI your coding standards and conventions
3. **Skills** give AI step-by-step workflows to follow
4. **Inline Prompts** provide the specific task at hand

Result: AI operates with full context — knows the project, follows standards, executes proven workflows, and focuses on your current task.
Key Insight: Without skills, you re-explain workflows every session. Without memory banks, you re-explain context. Without Agents.MD, you re-explain conventions. **Context engineering eliminates repetition.**

Verified Impact Data

Research-Backed Evidence for Context Persistence

The Context Overhead Problem

Industry Research (2024-2025)

- Developers spend **15-30 minutes/day** re-explaining project context to AI
- **32% of developer time** wasted reconstructing lost context
- **41% more bugs** in AI code lacking full system context

Sources: METR Study (2025), Uplevel Study (2024)

Context Persistence Solutions

Verified Performance Gains

Anthropic Prompt Caching (2024):

- **85% latency reduction** (time to first token)
- **90% cost reduction** for cached input tokens
- ROI positive after just **2 calls** to cached context

GitHub Copilot Optimization (2024):

- **20-35% cycle time reduction** from prompt optimization
- **55% faster** task completion with proper context
- Acceptance rates improved from 20% → 35%

Projected Team Impact

Conservative Model (45-developer team)

- Baseline: 20 min/day context overhead per developer
- Reduction: 70% (conservative vs. 85% research data)

: 14 min/dev/day × 45 devs = 262.5 hours/month

$26,000/month at $100/hour blended rate

Sources: Anthropic (Aug 2024), Peng et al. (2023) arXiv:2302.06590, GitHub Research (2024)

Common Mistakes to Avoid

❌ **Anti-Pattern 1: The Everything File**

- **Mistake:** Single 10,000-line `memory.md` file with all knowledge
- **Problem:** AI can't efficiently retrieve relevant context
- **Fix:** Modular files (architecture, conventions, domain separate)

❌ **Anti-Pattern 2: Code Duplication**

- **Mistake:** Copying entire modules into memory bank
- **Problem:** Quickly outdated, high maintenance, inefficient
- **Fix:** High-level patterns and principles, link to source

❌ **Anti-Pattern 3: Outdated Information**

- **Mistake:** "We use MongoDB" (team migrated to PostgreSQL 6 months ago)
- **Problem:** AI generates code for wrong tech stack
- **Fix:** Quarterly review and update process, version control

❌ **Anti-Pattern 4: Secrets in Memory**

- **Mistake:** "API key: abc123xyz" in memory bank
- **Problem:** Security risk, especially if memory banks are committed to repo
- **Fix:** Never store secrets, use environment variables

❌ **Anti-Pattern 5: Over-Prescription**

- **Mistake:** "Use exactly this algorithm with these parameters..."
- **Problem:** Removes AI's problem-solving ability
- vide principles and constraints, let AI implement

Live Demo: Setting Up Memory Banks

What We'll Build: Memory banks for the Task Manager project, including:

1. Architecture overview
2. Coding conventions
3. Domain glossary
4. Development workflow

Follow Along:

- Demo files will be available in course materials
- You'll create similar structure in lab
- Ask questions anytime

Demo Duration: 25 minutes **Q&A:** 5 minutes

What We Just Built

Memory Bank Structure Created:

```
memory-banks/
├── README.md                    ✅ Created
├── architecture/
│   └── overview.md              ✅ Created
├── conventions/
│   └── coding-standards.md      ✅ Created
├── domain/
│   └── glossary.md              ✅ Created
└── workflows/
    └── development-process.md   ✅ Created
```

Key Takeaways from Demo:

1. Start simple—4 files, ~5 pages total
2. Use AI to help create memory banks (meta!)
3. Test by generating code with/without memory
4. Iterate based on AI output quality

Next: You'll create memory banks for your project

Hands-On: Create Memory Banks for Your Project

Lab Structure:

Phase	Activity	Duration
1	Create memory bank structure	15 min
2	Write architecture overview	20 min
3	Write conventions and domain	25 min
4	Write workflow documentation	15 min
5	Test AI outputs (with/without memory)	15 min
6	Peer review and refinement	20 min

Total: 110 minutes (includes buffer time)

Deliverables:

- [ ] Memory bank structure (4-6 files)
- [ ] Architecture overview
- [ ] Coding conventions
- [ ] Domain glossary
- [ ] At least one workflow document
- [ ] Before/after comparison of AI outputs

Phase 1: Create Memory Bank Structure (15 min)

Step 1: Create folder structure

```
mkdir -p memory-banks/{architecture,conventions,domain,workflows,roles}
cd memory-banks
touch README.md
touch architecture/overview.md
touch conventions/coding-standards.md
touch domain/glossary.md
touch workflows/development-process.md
```

Step 2: Write README.md with navigation

```
# Memory Banks - [Your Project Name]

## Structure
- `architecture/`: System design, tech stack, deployment
- `conventions/`: Coding standards, testing patterns
- `domain/`: Business terms, rules, personas
- `workflows/`: Development, review, deployment processes
- `roles/`: Role-specific context (dev, QA, PM)

## Quick Links
- [Architecture Overview](architecture/overview.md)
- [Coding Standards](conventions/coding-standards.md)
- [Domain Glossary](domain/glossary.md)
- [Development Workflow](workflows/development-process.md)
```


Checkpoint: You have folder structure and README

Phase 2: Write Architecture Overview (20 min)

In `architecture/overview.md`, document:

1. System Architecture

- Architecture pattern (monolith, microservices, serverless, etc.)
- Key components and their interactions
- Diagram (optional but helpful)

2. Tech Stack

- Languages and versions
- Frameworks and major libraries
- Database(s) and data stores
- Infrastructure (cloud provider, containers, etc.)

3. Deployment

- Deployment approach (CI/CD pipeline)
- Environments (dev, staging, production)
- Key infrastructure details

Use AI to help: Ask AI to draft based on your Module 02 PRD and Agents.MD

✅ **Checkpoint:** Architecture overview complete (1-2 pages)

Phase 3: Conventions and Domain Knowledge (25 min)

In `conventions/coding-standards.md` **:**

- Pull from your Agents.MD (Module 02)
- Add any new conventions discovered since
- Include: naming, file structure, comments, testing

In `domain/glossary.md` **:**

- Define key domain terms
- Explain business rules
- Reference user personas from PRD

Example Domain Entry:

```
## Bolt
A short development cycle (hours to days, not weeks). Replaces traditional
2-week sprints in AI-native development. Named after AWS AI-DLC methodology.

## Async-First Workflow
Development approach optimized for distributed teams across time zones.
Emphasizes asynchronous communication and documentation over synchronous meetings.
```

✅ **Checkpoint:** Conventions and domain files complete

Phase 4: Development Workflow (15 min)

In `workflows/development-process.md` **:**
Document how your team works. Include:
1. Development Workflow

- How features go from idea to production
- Branch strategy (Git flow, trunk-based, etc.)
- PR process and review requirements

2. Code Review Checklist

- What reviewers look for
- Definition of "done"
- Common issues to avoid

3. Testing Strategy

- Test types required (unit, integration, E2E)
- Coverage targets
- When to write tests (TDD, after implementation, etc.)

Keep it concise: 1-2 pages max. Focus on what AI needs to know.
✅ **Checkpoint:** Development workflow documented

Phase 5: Test Memory Banks (15 min)

Test AI Output Quality:

Step 1: Choose a task from your Module 02 User Stories

Step 2: Generate code WITHOUT memory banks loaded

- Use generic prompt
- Save the output

Step 3: Generate code WITH memory banks loaded

- Use same prompt
- Load memory banks into AI context
- Save the output

Step 4: Compare

- Does code with memory banks follow your conventions?
- Does it use your tech stack correctly?
- Does it reflect your architecture?
- Is domain terminology correct?

Document the difference - You'll share in peer review

✅ **Checkpoint:** Before/after comparison complete

Phase 6: Peer Review and Refinement (20 min)

Peer Review Format:

Find a partner or group of 3

Each person shares (5 min per person):

1. **Show your memory bank structure**
   - What files did you create?
   - What content did you prioritize?
2. **Show before/after AI outputs**
   - What improved with memory banks?
   - What still needs work?
3. **Challenges encountered**
   - What was difficult to document?
   - What are you unsure about?

Peer reviewer gives feedback:

- ✅ What's working well
- 🔧 Suggestions for improvement
- ❓ Clarifying questions

Refine based on feedback (final 5 min)

✅ **Checkpoint:** Peer review complete, refinements made

What Did We Learn?

Group Discussion:

1. **What surprised you about memory banks?**
   - Harder or easier than expected?
   - Bigger or smaller impact than expected?
2. **What was most valuable to document?**
   - Which memory bank file had biggest impact?
   - What context mattered most?
3. **What will you do differently in real work?**
   - Will you implement memory banks at work?
   - What changes will you make?
4. **What questions remain?**
   - What's still unclear?
   - What do you need more guidance on?

Format: Open discussion, 10-15 minutes

What We Learned in This Module

1. Context Engineering Matters

- AI assistants are stateless by default—they forget between sessions
- Well-engineered context improves AI accuracy by 40-60%
- ROI is measurable and significant

2. Memory Banks Provide Persistent Context

- Structured knowledge repositories for project-specific context
- Four types: Episodic, Semantic, Procedural, Working
- Focus on semantic (architecture, domain) and procedural (workflows, conventions)

3. Start Simple, Iterate

- Begin with 4-5 files, 5-10 pages total
- Architecture, conventions, domain, workflows
- Test, refine, expand based on needs

4. Memory Banks + Agents.MD + Prompts

- Memory banks provide broad context
- Agents.MD provides coding rules
- Inline prompts provide task details

 l three together for best results

5. Maintenance is Manageable

Continuing Your Learning

Before Module 06 (SpecKit Framework):

Required:

- [ ] Review your memory banks from this lab
- [ ] Refine based on peer feedback
- [ ] Test with 2-3 more AI generation tasks

Optional (Highly Recommended):

- [ ] Implement memory banks for a real work project
- [ ] Track before/after metrics (time saved, rework reduced)
- [ ] Share with your team and get feedback

Preparation for Module 06:

- Bring your PRD, Epics, Stories (Module 02)
- Bring your memory banks (Module 03)
- We'll use SpecKit to orchestrate the entire flow

Resources:

- Memory bank templates in course materials
- Example memory banks from demo
- ımended reading list (in course portal)

Questions?

Open Q&A

Common Questions We Can Address:

- Implementation details for specific tools
- Scaling memory banks to larger teams
- Handling frequent architecture changes
- Security and compliance considerations
- Integration with existing documentation

Your Questions:

- ???

Great Work! 🎉

You've Accomplished: ✅ Understanding context engineering fundamentals ✅ Designing and implementing memory bank structure ✅ Creating project-specific memory banks ✅ Testing AI output improvements ✅ Peer review and refinement

Impact:

- Your AI assistant now has persistent project knowledge
- You'll save 15-30 minutes per day on context explanations
- AI-generated code will better match your project standards

Coming Soon (Module 06):SpecKit Framework

- GitHub's open-source spec-driven development toolkit
- Orchestrating Specify → Plan → Tasks → Implement
- Integrating PRD (Module 02) + Memory Banks (Module 03)

See you next time!